

Controlling Transactions and Locks in SQL - Part 1

By Don Schlichting

Controlling Transactions and Locks with Hints and Snapshots in SQL.

Locks

Locks are the mechanism databases use to control problems that may arise from simultaneous multi-user access. Some common issues locks try to manage include; what happens when two users try to change the same piece of information at the same time - who wins? Or user one tries to read data while user two is making changes on the same information, - which version of the data should user one see, changed or unchanged? Without locks, these situations may cause the data or results of a query to be logically incorrect. Lock sophistication is a key difference between simple databases designed for single user applications and enterprise databases capable of simultaneously handling large volumes of multi-user actions. Locks are automatically created and managed by Microsoft SQL, but there are cases when a developer or DBA will obtain better performance by dictating the locking behavior SQL should use. This article will introduce Transactions and Locks, as well as methods for controlling them such as using Locking Hints and Snapshots.

Before a review of locks can take place, transactions (the smallest unit of work in TSQL), must be introduced.

Transactions

Think of a transaction as a logical container holding TSQL that can stand alone, not needing any other statements to be complete. Every transaction causes at least one lock to be issued. If you issue a simple one-statement update such as:

```
UPDATE authors  
SET au_fname = 'Don'
```

Then your transaction consists of only this one statement. However, if the above statement first required that some new records be inserted into the table, then the transaction would consist of two statements:

```
INSERT INTO authors  
(au_fname)  
SELECT au_fname  
FROM some_other_table  
UPDATE pubs  
SET first_name = 'Don'
```

To tell SQL that these two statements must happen together, in the order presented, they are both wrapped together inside a single transaction, noted by the key phrase BEGIN TRANSACTION:

```
BEGIN TRANSACTION  
INSERT INTO authors
```

```
(au_fname)
SELECT au_fname
FROM some_other_table
UPDATE pubs
SET first_name = 'Don'
COMMIT
```

COMMIT

The COMMIT tells SQL that it has reached the end of the transaction, and whatever data modifications the statements have made should become a permanent part of the database. The other option for ending a transaction is ROLLBACK, which means all the changes just made should be undone. At this point, for both COMMIT and ROLLBACK, all resources and locks used by the transaction are released.

As transactions become longer and more involved, locking becomes more problematic. The code below will create a simple example of this. We will simulate a long running transaction by not issuing a COMMIT; this will hold a lock on records a second statement will request. In SQL, run the following TSQL from Query Analyzer:

```
USE pubs
GO
BEGIN TRANSACTION
  UPDATE authors
  SET au_fname = 'Don'
```

Now open a second instance of Query Analyzer and run this select:

```
SELECT au_fname
FROM authors
```

The select query will not get a response. Its waiting for the lock SQL automatically issued at the BEGIN TRANSACTION to be released. To release it, issue this key word from the first Query Analyzer session:

```
ROLLBACK
```

When you do, the update changes will be discarded and the second Query Analyzer session will get a dataset back.

Transactions and locks increase the complexity of database operations, but they guarantee valid data and query results in multi-user applications. This guarantee is expressed with the acronym ACID. ACID stands for Atomicity, Consistency, Isolation, and Durability.

For a transaction to be Atomic, all of the DML statements (Data Manipulation Language, INSERT, UPDATE, and DELETE) must either all commit, or rollback. Meaning a transaction cannot be left in a half done state. An example would be a transfer of funds from a savings account to checking. If the withdrawal from saving happens, the credit to checking must happen, or if the withdrawal fails, then the credit to checking should not happen. In either case, both must either succeed or fail together.

Consistency means a user should never see data changes in mid transaction. Their view should return the data as it appeared prior to beginning the transaction, or if the transaction is finished, then they should see the changed data. Using the above checking account example, a select run at the same time as the transfer transaction should not return the saving account balance debited, but the checking account not yet credited.

Isolation is at the heart of multi-user transactions. It means one transaction should not disrupt another. In the transfer example, if at the same time we are transferring money from savings to checking, our employer is making a payroll direct deposit into the same checking account, neither transaction should cause the other to corrupt data.

Durability implies that the changes made by the transaction are recorded permanently.

Lock Granularity

The simplest way to meet the ACID requirement would be for a transaction to create a lock that seizes the entire database being affected. Not letting anyone else touch or see any other data until the transaction completed. This solution would of course cause a great deal performance problems as other transactions tried to access data. The other extreme would be to lock on a single row. Alternatively, if the transaction needed a million rows, issue a million locks, one on each row. This would leave most of the database open to other transactions, but each lock issued takes up memory on the server. So even though locking on a row-by-row basis would leave most of the database open, locking a million rows would generate quit a bit of overhead and performance again would suffer. To avoid this, locks usually try to seize a group of something right sized for the transaction. A lock can be issued against a Database, Table, Extent, Page, or a Row. (A Page is usually an eight-kilobyte storage area on the hard drive. An Extent is a group of eight pages.) This type of right sizing will give the best performance with the least amount of restriction.

Conclusion

Locks are occurring on the database at all times. Every Select and DML statement issues a lock. Although SQL can control locks automatically, sometimes controlling them manually will lead to increased performance. Next moth, we will introduce a variety of different locks and methods for controlling or requesting them.